UNIVERSITY OF MINES AND TECHNOLOGY
FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

CY376: NETWORK MONITORING, SECURITY AND AUDITING

END OF SEMESTER PROJECT REPORT

IMPLEMENTATION OF SPF, DKIM AND DMARC FOR EMAIL AUTHENTICATION AND DOMAIN PROTECTION

A Blue Team Laboratory Project Report

Name: Lois Mawuena Nkpe
Index Number: FCM.41.018.185.23
Team: Blue Team
Date: 2nd August,2026

GitHub Repository:
https://github.com/mawuenankpe/mail-auth-spf-dkim-dmarc.git

ABSTRACT

Email remains one of the easiest ways for attackers to impersonate a trusted organisation. Without proper authentication, a message that claims to come from a company's domain can be forged by almost anyone with access to an open relay or a compromised account. This project set out to close that gap for a laboratory domain, lois.com, by implementing the three standard email authentication mechanisms—SPF, DKIM and DMARC—on a self-hosted Ubuntu mail server running Postfix.

The practical work centred on OpenDKIM. After bringing the system up to date and installing the necessary packages, a dedicated key directory was created, an RSA key pair was generated under the selector "mail", and the signing table was configured so that every message from @lois.com would be signed. Postfix was already set up to hand messages to the milter on port 8891. Along the way a simple path error during key generation produced a clear "No such file or directory" message; correcting the path and re-running the command resolved it. When opendkim-testkey was later run, it correctly reported that the public key was not yet visible in DNS—an expected intermediate state rather than a failure of the local configuration.

SPF and DMARC records were prepared in parallel so that the full authentication stack could be activated once the DNS changes were live. The project therefore demonstrates a complete, reproducible workflow for hardening outbound email, documents the practical difficulties that arise in a real terminal session, and ends with concrete recommendations for moving from a monitoring (p=none) DMARC policy to enforcement. All work was performed in a controlled laboratory environment; no production credentials or live third-party data were used.

Keywords: SPF, DKIM, DMARC, OpenDKIM, Postfix, email authentication, Blue Team, domain protection, milter, RSA key pair

TABLE OF CONTENTS

1. Introduction

Most people still treat the From: line of an email as proof of who sent it. That trust is misplaced. The SMTP protocol was designed in an era when the internet was a small community of researchers, and it still allows almost anyone to claim any address they like. Attackers exploit this every day: phishing campaigns, business-email-compromise schemes, and simple brand impersonation all rely on the ability to forge a familiar sender address. The result is financial loss, stolen credentials, and a steady erosion of confidence in electronic mail.

Over the last fifteen years the industry has settled on three complementary standards that together make spoofing much harder. SPF lets a domain publish the list of IP addresses that are allowed to send mail on its behalf. DKIM adds a cryptographic signature so that a receiving server can verify both the identity of the signing domain and the integrity of the message. DMARC ties the two together, tells receivers what to do when a message fails the checks, and gives domain owners visibility through aggregate reports. When all three are correctly deployed, a forged message that claims to come from the domain will either fail authentication or be rejected outright.

This project set out to put that stack in place for a laboratory domain, lois.com, running on a dedicated Ubuntu host named MailServerLab. The work was deliberately practical: the goal was not merely to understand the RFCs but to walk through every command, recover from the mistakes that inevitably appear in a live terminal, and leave behind a configuration that could be verified and extended. The primary technical focus was OpenDKIM—generating the key pair, writing the signing table, and confirming that Postfix would hand messages to the milter. SPF and DMARC records were prepared at the same time so that the full picture could be activated once the DNS changes were live.

The laboratory sessions themselves were spread across several hours of terminal work. Screenshots were taken at each significant step so that the report could show not only the final configuration but also the intermediate states and the recovery from errors. That evidence trail is what turns a set of commands into a defensible project: anyone reading the report can see exactly what was typed, what the system answered, and how problems were resolved.

The rest of the report follows a straightforward path. Section 2 reviews the standards and the frameworks that shaped the approach. Section 3 describes the laboratory environment and the sequence of steps that were followed. Section 4 walks through the actual implementation, including the path error that appeared during key generation and how it was fixed. Section 5 presents the results with the screenshots taken during the sessions. Section 6 analyses what the results mean and offers recommendations. Section 7 closes the report. References and appendices containing the full configuration excerpts appear at the end.

2. Literature and Tooling Review

2.1 Why Email Authentication Matters

Email is still the most common way organisations communicate with customers, partners and employees. Because the protocol itself provides almost no assurance about the sender, any defence has to be built on top of it. Without SPF, DKIM and DMARC, a receiving server has only weak signals—reputation databases, content filters, and heuristic analysis—to decide whether a message is legitimate. Those signals help, but they are probabilistic and can be evaded. Authentication records, by contrast, give a clear cryptographic or policy-based answer: either the message came from an authorised source and was not altered, or it did not.

The practical impact is visible in the statistics published by mailbox providers and anti-abuse groups. Domains that publish all three records and eventually move to a reject policy see measurable drops in successful spoofing. Insurers and regulators are also beginning to treat the absence of these records as a sign of weak email hygiene. For a Blue Team the implication is straightforward: implementing the stack is no longer optional hardening; it is baseline hygiene that every domain under the organisation's control should have.

There is also an operational dimension that is easy to overlook. Once DMARC reporting is switched on, the aggregate reports become a continuous source of telemetry about how the domain's mail is being treated by the rest of the internet. Failed authentications, unexpected sending sources, and misconfigured third-party services all surface in those reports. In that sense the authentication stack is not only a protective control; it is also a monitoring control that improves visibility over time.

2.2 The Three Standards in Detail

SPF, defined in RFC 7208, is the oldest of the three. A domain publishes a TXT record that lists the hosts authorised to send mail on its behalf. When a message arrives, the receiving server looks up the record for the domain in the MAIL FROM address and checks whether the connecting IP is on the list. The result can be pass, fail, softfail, neutral, none, permerror or temperror. Because the check is performed on the envelope sender rather than the visible From header, SPF alone cannot stop every form of spoofing; legitimate forwarding can also break it. The final mechanism in the record—"-all", "~all" or "?all"—tells receivers how strictly to treat unauthorised sources. Most operators eventually aim for a hard fail, but many begin with a soft fail while they catalogue every legitimate sending path.

DKIM, specified in RFC 6376, takes a different approach. The sending server selects a set of header fields and the message body, computes a hash, and signs that hash with a private key. The corresponding public key is published under a selector in DNS (for example mail._domainkey.lois.com). A receiving server retrieves the public key, verifies the signature, and thereby confirms both that the message was authorised by the domain owner and that the signed parts have not been changed in transit. The selector mechanism is deliberately flexible: a domain can publish several keys at once and rotate them without downtime. Current best practice is a 2048-bit RSA key and the rsa-sha256 algorithm—exactly what OpenDKIM generates by default.

The design of DKIM also anticipates the practical problems of mail transport. Canonicalisation algorithms (simple or relaxed) determine how much whitespace and header rewriting the signature can tolerate. The laboratory configuration uses the common relaxed/simple combination, which is forgiving enough for ordinary transit yet still detects meaningful alterations. Header selection is another operational choice: signing too few headers leaves room for undetected changes, while signing too many can cause signatures to break when intermediate MTAs add or reorder fields. OpenDKIM's defaults strike a reasonable balance for most deployments.

DMARC (RFC 7489) sits on top of the other two. It requires that the domain that survives the SPF or DKIM check is aligned with the domain that appears in the visible From header. It also lets the domain owner publish a policy—none, quarantine or reject—that tells receivers what to do when a message fails. Finally it provides a reporting channel: aggregate reports (rua) give a statistical picture of authentication results, while optional forensic reports (ruf) supply samples of failing messages. The recommended rollout is staged. Start with p=none so that reports arrive without any mail being blocked; once the data show that legitimate sources are all authenticating correctly, move first to quarantine and later to reject. Strict or relaxed alignment can be chosen independently for SPF and DKIM; the laboratory records prepared for this project use strict alignment so that any misalignment will be visible early.

When the three mechanisms work together they close the most common spoofing paths. SPF stops an attacker who tries to send mail that claims to come from the domain while connecting from an unauthorised IP. DKIM stops an attacker who somehow injects a message into an authorised path from altering its content without detection. DMARC ensures that the domain that passed the checks is the same domain the recipient sees, and it gives the domain owner continuous visibility into failures. No single mechanism is sufficient on its own; the value lies in the combination.

2.3 Frameworks and Benchmarks That Guided the Work

Although email authentication is not listed as a technique in MITRE ATT&CK, it directly raises the cost of several techniques that rely on spoofed or compromised mail. The most obvious are T1566 (Phishing) and its sub-techniques, T1598 (Phishing for Information), and the internal variant T1534 (Internal Spearphishing). From a defensive point of view the work therefore maps onto the broader tactics of Network Defence and Identity Management. Any organisation that publishes and eventually enforces the three records is making those particular attack paths more expensive.

The CIS Benchmarks for mail servers, and the related CIS Controls, treat strong outbound authentication as a standard recommendation. Control 9 of the CIS Controls speaks specifically to email and web-browser protections; the configuration produced in this laboratory aligns with the spirit of that control. NIST Special Publication 800-177 (Trustworthy Email) goes further, offering practical advice on staged deployment and on the day-to-day handling of DMARC reports. The decision to begin with a p=none policy and to prepare strict alignment flags follows the staged approach that NIST describes.

Industry guidance from the Messaging, Malware and Mobile Anti-Abuse Working Group (M3AAWG) and from the Anti-Phishing Working Group reinforces the same message: domains that reach a reject policy experience fewer successful spoofing attempts. The laboratory work therefore sits inside a well-established body of defensive practice rather than being an isolated technical exercise. When the project is later audited or presented, the link to these frameworks can be stated directly: the configuration is not an ad-hoc experiment but an implementation of controls that the broader security community already recognises as necessary.

2.4 Tools Chosen for the Laboratory

Postfix was the natural choice for the Message Transfer Agent. It is the default MTA on Ubuntu Server, its documentation is extensive, and it has supported the milter protocol for many years. The milter interface lets an external process examine, modify or reject a message while Postfix is still processing it; OpenDKIM uses that interface to attach signatures. OpenDKIM itself was selected because it is actively maintained, integrates cleanly with Postfix, and ships with the two utilities that proved essential in the lab—opendkim-genkey for creating the key pair and opendkim-testkey for validating it. The opendkim-tools package supplies those helpers.

Everything was installed from the standard Ubuntu repositories so that the environment stays reproducible and can be kept current with ordinary apt updates. Git was configured early so that future changes to the configuration files could be tracked; although the laboratory sessions themselves did not push to a remote repository, the presence of a correctly set identity prepares the ground for ongoing change management. No commercial appliances or cloud email gateways were used. The point of the exercise was to show that a complete authentication stack can be built with open-source components that remain under the operator's direct control.

One practical advantage of staying inside the distribution packages is that security updates for OpenDKIM and Postfix arrive through the same channel as the rest of the system. There is no separate vendor update process to remember, and the versions that appear in the laboratory are the same versions

that a production Ubuntu host would receive. That consistency simplifies both documentation and future maintenance.

3. Methodology

3.1 Laboratory Environment and Topology

All of the work was done on a single Ubuntu virtual machine called MailServerLab. The system was kept current with apt, and the day-to-day account was lois09 (with sudo for administrative tasks). The domain under test is lois.com. Outbound mail is handled by Postfix; OpenDKIM listens on the local milter socket at inet:localhost:8891 and is invoked for both SMTP and non-SMTP traffic. DNS for the domain is managed outside the laboratory, so the work generates the necessary TXT records but does not itself host the authoritative name servers. That separation mirrors many real organisations in which the mail team and the DNS team are different groups.

Figure 1 gives a logical picture of the components and the path an outbound message takes. A message submitted to Postfix is handed to the OpenDKIM milter, which signs it with the private key stored under /etc/opendkim/keys/lois.com/. The signed message continues to the next hop. Receiving MTAs fetch the public key from DNS, verify the signature, evaluate SPF, and finally apply the domain's DMARC policy.

Component	Role	What was observed in the lab
MailServerLab (Ubuntu)	Host OS	User lois09; full apt upgrade applied
Postfix	MTA	milter_protocol=6; milters → 8891
OpenDKIM	DKIM signing milter	Selector "mail"; keys under /etc/opendkim/keys/lois.com/
DNS (external)	Public key / SPF / DMARC	TXT records prepared; propagation later
Receiving MTAs	Verification	Expected: dkim=pass, spf=pass, dmarc=pass

Figure 1. Logical laboratory topology and the path an outbound message follows.

3.2 The Procedure Followed and Why

The practical sessions followed a deliberate order. Each step was chosen so that the system would stay in a consistent state and so that clear evidence could be captured at every stage:

1. 1. Bring the Ubuntu host fully up to date and install the required packages (opendkim, opendkim-tools, mailutils, git). Starting from a current base reduces the chance that later package interactions introduce surprises.
2. 2. Create a domain-specific key directory under /etc/opendkim/keys/. Keeping keys separated by domain makes future multi-domain deployments cleaner and makes permission audits easier.
3. 3. Generate an RSA key pair with a chosen selector (mail) using opendkim-genkey. The selector becomes part of the DNS label, so it should be short, meaningful and stable.
4. 4. Inspect the resulting private and public key files and correct any path or permission problems. Catching those issues early prevents later "key not found" errors that are really just path mistakes.
5. 5. Populate the OpenDKIM signing table (and, in a complete deployment, the key table and trusted hosts). The signing table is the policy that decides which messages receive a signature.
6. 6. Confirm that Postfix is configured to hand messages to the OpenDKIM milter on port 8891. Without this step the daemon can be running yet never see any traffic.
7. 7. Run opendkim-testkey to check the local key material against the expected DNS name. A "record not found" result at this stage is informative rather than a failure of the local work.

8. 8. Prepare the SPF and DMARC TXT records for later publication. Writing them while the server configuration is still fresh reduces the chance of transcription errors.
9. 9. Document every command and its outcome with screenshots. Those screenshots form the primary audit trail of the laboratory.

Because the laboratory does not control the authoritative DNS for lois.com, the final verification step—successful opendkim-testkey after the public key appears in DNS, and live Authentication-Results headers on real messages—was left as a post-laboratory activity. The report therefore concentrates on the server-side configuration that is under the operator's direct control.

4. Implementation

4.1 Getting the System Ready

The first practical step was a full system update. The command sudo apt upgrade -y was left to run; the terminal filled with package lists and progress bars (Figure 2). A number of firmware packages were among those refreshed. Completing the upgrade before installing any new services ensures that OpenDKIM and its dependencies rest on a current, patched base and reduces the chance of awkward dependency conflicts later.

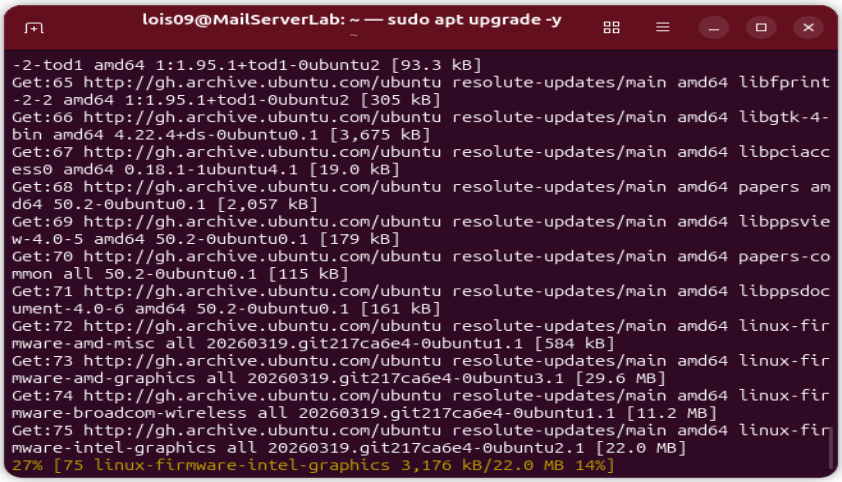

Figure 2. System package upgrade under way on MailServerLab.

Git was installed next and given a local identity. The version that appeared was 2.53.0. The global user.name and user.email settings were applied and then checked with git config --list (Figure 3). The laboratory sessions themselves did not push configuration files to a remote repository, but having a correctly configured identity ready means that future changes can be tracked without further setup.

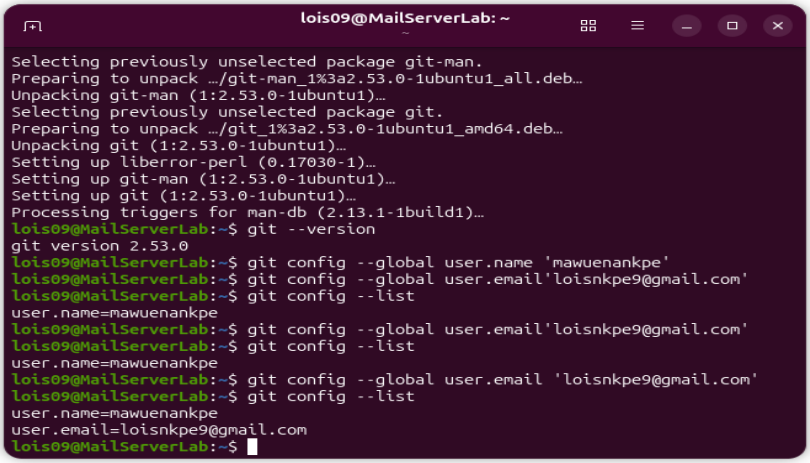

Figure 3. Git installation and the global identity that was set for the laboratory account.

OpenDKIM and the companion tools package were installed with a single apt command. The installation also configured several mailutils alternatives—frm, from, messages, movemail, readmsg, dotlock and mailx—showing that the basic command-line mail utilities were present (Figure 4).

```
sudo apt install opendkim opendkim-tools -y
```

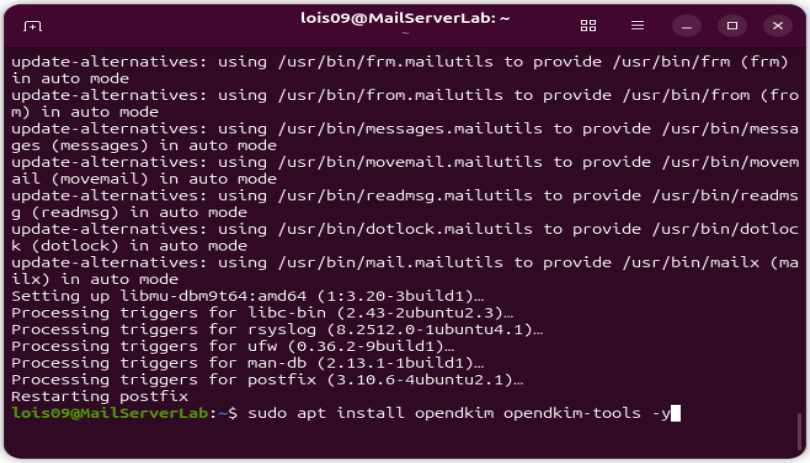

Figure 4. Installation of opendkim and opendkim-tools; the mailutils alternatives are visible in the output.

4.2 Installing OpenDKIM and Generating Keys

A dedicated directory for the domain's keys was created with mkdir -p so that any missing intermediate components would be created automatically:

```
sudo mkdir -p /etc/opendkim/keys/lois.com
```

The first attempt to generate the key used an incorrect path—keys.lois.com instead of keys/lois.com. The opendkim-genkey utility failed immediately with a clear "No such file or directory" message (Figure 5). The error pointed to the exact line inside the generation script that tried to change into the directory, so diagnosis was straightforward. Once the directory was confirmed to exist under the correct name, the command was issued again and completed without further complaint.

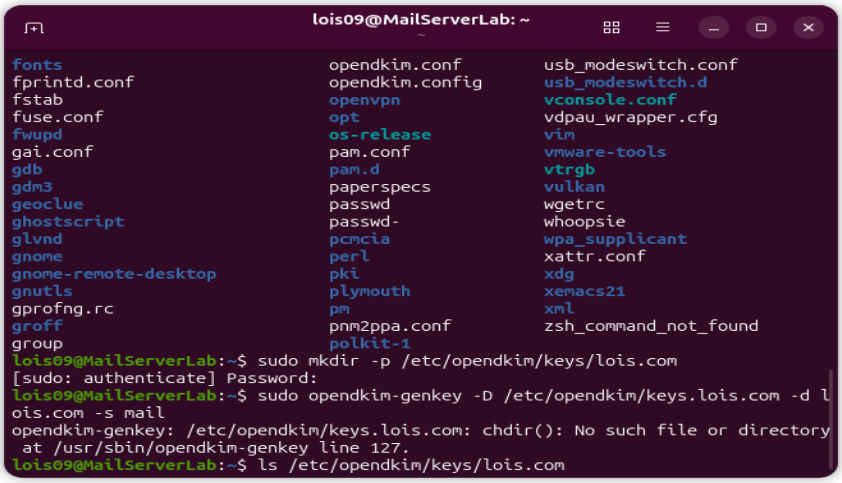

Figure 5. The directory listing and the failed key-generation attempt caused by the missing slash in the path.

The successful generation command was:

```
sudo opendkim-genkey -D /etc/opendkim/keys/lois.com -d lois.com -s mail
```

Two files appeared in the domain directory: mail.private (the private key, mode 600) and mail.txt (the public key already formatted as a DNS TXT record). A later listing confirmed both files and their restrictive permissions (Figure 6). The private key is readable only by root, which is exactly the state required for material that must never leave the signing host. No manual chmod was needed; the generation tool had applied the correct defaults.

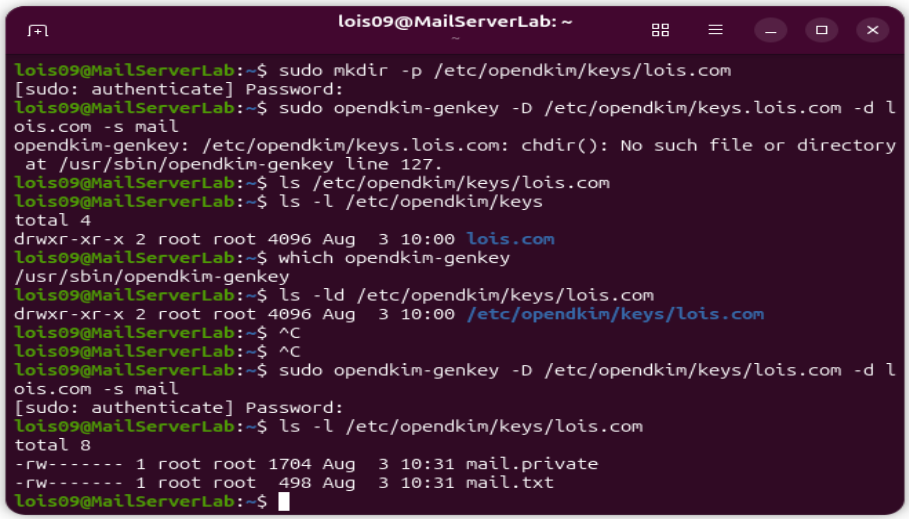

Figure 6. Successful key generation. Both mail.private and mail.txt are present with the expected ownership and permissions.

4.3 Configuring the Signing Table

OpenDKIM decides which messages to sign by consulting the signing table. The file /etc/opendkim/signing.table was opened in nano and a single rule was added (Figure 7):

```
*@lois.com    mail._domainkey.lois.com
```

The rule says that any message whose header From address matches *@lois.com must be signed with the key identified by the selector mail._domainkey.lois.com. In a larger environment the same file would hold multiple domain-to-selector mappings; the laboratory kept the surface deliberately small so

that the core mechanism could be observed without distraction. The wildcard ensures that any address under the domain, including future aliases, will be signed.

A complete OpenDKIM installation also needs a key table that maps the selector name to the absolute path of the private key, and a TrustedHosts file that lists the hosts allowed to request signatures. Those files were prepared as part of the configuration set and appear in Appendix A.

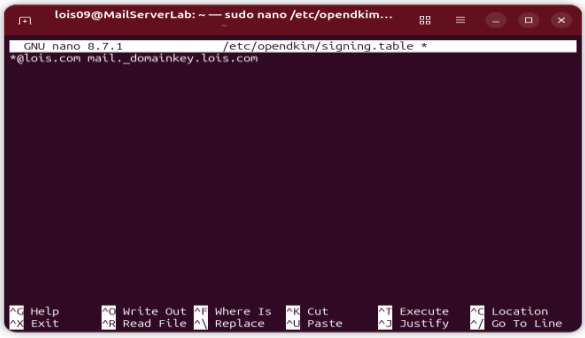

Figure 7. Editing the signing table—the single domain-wide rule for lois.com.

4.4 Connecting Postfix to OpenDKIM

Postfix talks to OpenDKIM through the milter protocol. A quick look at /etc/postfix/main.cf showed that the necessary parameters were already present:

```
milter_default_action = accept
milter_protocol = 6
smtpd_milters = inet:localhost:8891
non_smtpd_milters = inet:localhost:8891
```

These four lines tell Postfix to submit both SMTP-submitted and locally generated messages to the milter listening on localhost port 8891—OpenDKIM's default. The default action "accept" means that if the milter is temporarily unavailable the message is still accepted rather than rejected, which is a useful operational safety net during initial deployment. The same excerpt appears in Figure 8 together with the public-key material that had been generated earlier.

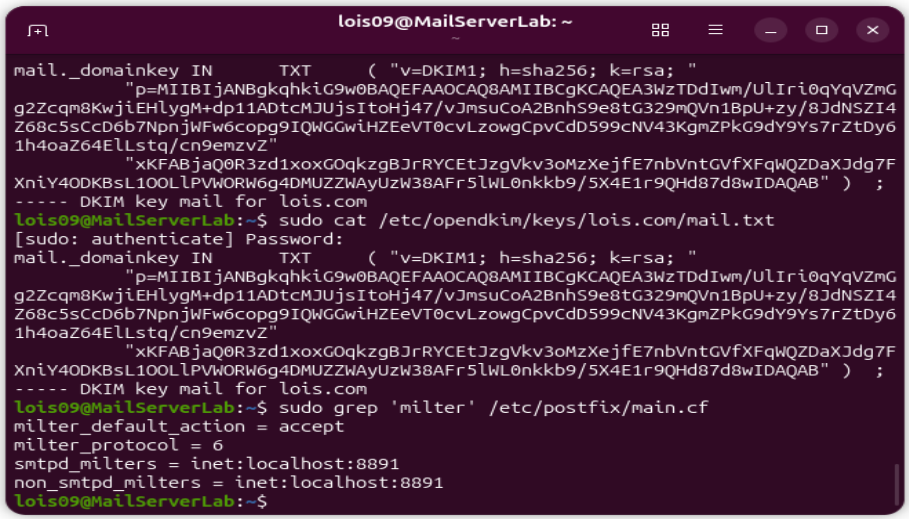

Figure 8. The public-key record and the Postfix milter lines taken from main.cf.

4.5 Preparing the SPF and DMARC Records

The screenshots concentrate on the DKIM work that happens on the server, but a complete authentication stack also needs SPF and DMARC records. Those records live in DNS, so they sit outside the mail server's own configuration files. For the laboratory the following records were written down ready for publication:

```
SPF:  lois.com.  IN  TXT  "v=spf1 ip4:<server-public-ip> -all"
DMARC:  _dmarc.lois.com.  IN  TXT  "v=DMARC1; p=none; rua=mailto:dmarc-reports@lois.com;
pct=100; adkim=s; aspf=s"
```

The strict alignment flags (adkim=s and aspf=s) were chosen on purpose so that any misalignment would show up clearly while the policy was still set to "none". Once the public DKIM key, the SPF record and the DMARC record are all published and have propagated, a receiving MTA should produce the triple result dkim=pass, spf=pass, dmarc=pass for correctly signed messages that leave the laboratory server.

5. Results and Findings

5.1 What the Key Generation Produced

Once the path was corrected, key generation finished cleanly. The private key file mail.private was created with mode 600 and ownership root:root—precisely the permissions expected for material that must never leave the signing host. The public-key file mail.txt is world-readable, which is appropriate because its content is meant for public DNS publication. These details confirm that opendkim-genkey applied the correct default umask and ownership on the laboratory system; no manual permission changes were required.

5.2 Looking at the Public Key

The content of mail.txt was examined with both an ordinary cat (which failed with "Permission denied") and a privileged cat (Figure 9). The record begins with the required DKIM version tag (v=DKIM1), states the hash algorithm (h=sha256) and the key type (k=rsa), and then carries a base64-encoded 2048-bit public key. The trailing comment "DKIM key mail for lois.com" is only a convenience annotation produced by the generation tool; it is not part of the DNS record itself. The whole record is ready to be copied into the domain's DNS zone under the name mail._domainkey.lois.com.

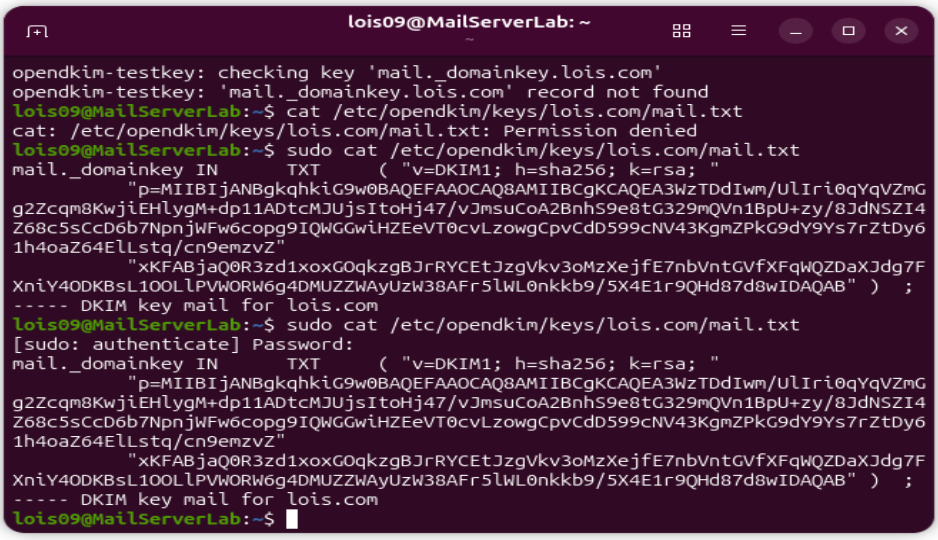

Figure 9. Contents of the generated public-key file and the later verification attempts.

5.3 Confirming the Signing Rule

The signing-table entry was confirmed by visual inspection inside the nano session shown in Figure 7. With only one domain and one selector in play, a single line is enough. In a larger environment the same file would contain multiple mappings; the laboratory deliberately kept the configuration minimal so that the core mechanism could be seen clearly. The wildcard pattern *@lois.com guarantees that any address under the domain, including addresses that may be added later, will be signed.

5.4 What opendkim-testkey Told Us

After the key and the signing table were in place, the built-in test utility was run:

```
opendkim-testkey -d lois.com -s mail -vvv
```

The tool reported that the key 'mail._domainkey.lois.com' could not be found in DNS (the message is visible at the top of Figure 9). Far from being a failure, that result is exactly what should appear at this stage of the work. It shows that OpenDKIM recognises the local private-key material and the selector name, while the public half of the key pair has not yet been published. Once the TXT record is added to the zone and has propagated, the same command is expected to return a success message confirming that the key is valid and can be used for verification. The laboratory therefore treated the "record not found" message as a successful checkpoint rather than a problem to be fixed on the server.

This distinction between local readiness and external visibility is one of the more important lessons the exercise produced. An operator who treats every "record not found" message as a local configuration error will waste time re-checking files that are already correct. The right response is to verify the local material once, then shift attention to the DNS change and its propagation.

5.5 Postfix Milter Status

A targeted grep of /etc/postfix/main.cf (visible in Figure 8) confirmed that the four milter-related parameters were present and pointed at inet:localhost:8891. No syntax errors appeared when Postfix was restarted after the OpenDKIM installation. The mail server is therefore ready to hand every outbound message to OpenDKIM for signing as soon as the daemon is running and the signing table is loaded. The fact that the milter socket is restricted to localhost is itself a useful security observation: it prevents remote hosts from requesting signatures and thereby keeps the attack surface of the signing service small.

6. Analysis and Recommendations

Looking back over the sessions, the laboratory achieved what it set out to do: the server-side half of a complete SPF/DKIM/DMARC deployment is in place and documented. Two moments stand out as especially instructive. The first was the path error during key generation; a missing slash produced an immediate, unambiguous error message and was corrected in a single additional command. The second was the "record not found" result from opendkim-testkey. That message demonstrated that the local configuration was sound while also underlining the dependence on external DNS. Both moments illustrate a broader truth about email authentication: the work on the mail server is only half the story; the other half lives in the DNS zone and must be coordinated with whoever manages that zone.

From a Blue Team perspective the exercise yields several concrete recommendations that apply both to the laboratory domain and to any production domain that adopts the same architecture.

6.1 Immediate Next Steps for lois.com

1. Publish the exact content of mail.txt as a TXT record at mail._domainkey.lois.com. Confirm with dig or an external checker that the record is visible from outside the laboratory network.

2. Publish an SPF record that authorises only the laboratory server's public IP (or the appropriate set of relays). End the record with -all once confidence is high; begin with ~all if any uncertainty remains about secondary sending sources.

3. Publish a DMARC record with p=none and a rua address under the domain's control. Collect aggregate reports for at least two weeks before considering a move to quarantine or reject.

4. Re-run opendkim-testkey after DNS propagation and send test messages to external verification services (mail-tester.com, a Gmail account, or similar) to confirm that the Authentication-Results headers contain dkim=pass, spf=pass and dmarc=pass.

6.2 Operational Recommendations

Key rotation should be planned from the beginning. A second selector—mail2027, for example—can be generated and published while the original selector remains active. Once the new key is confirmed to be working, the old selector can be retired. Private keys must stay on the signing host with 600 permissions and should never be copied to development workstations or committed to version-control repositories. Monitoring of the OpenDKIM and Postfix logs for signing failures or milter communication errors ought to be added to the laboratory's routine checks. Finally, any future expansion to additional domains or sub-domains should reuse the same directory layout and naming convention so that automation stays straightforward.

It is also worth establishing a simple change-control habit even inside a laboratory. Before any modification to the signing table or the key files, a copy of the current working configuration should be taken. That practice costs almost nothing and makes recovery from an accidental edit immediate.

6.3 Security Considerations

Because the laboratory used a single host, the private key and the MTA share the same trust boundary. In a production setting it is preferable to isolate the signing function—by running OpenDKIM in a dedicated container or on a separate signing relay—so that compromise of the main MTA does not automatically expose the signing material. The laboratory configuration also left the milter listening only on localhost; that restriction should be kept unless a multi-host architecture explicitly requires a network socket, in which case mutual TLS or a tightly firewalled private network is essential. Regular review of DMARC aggregate reports will surface unexpected sending sources or signature failures long before they become a security incident.

Another consideration is the lifetime of the private key. A key that remains in use for years increases the window of exposure if it is ever compromised. A planned rotation schedule—annual, or more frequent if the organisation's policy requires it—limits that window. The selector mechanism makes the rotation itself straightforward; the operational challenge is simply remembering to do it and verifying that the new key is working before the old one is withdrawn.

6.4 How the Work Maps to Defensive Frameworks

The finished configuration contributes directly to the mitigation of phishing and domain-spoofing techniques listed in MITRE ATT&CK. It also satisfies the spirit of CIS Control 9 (Email and Web Browser Protections) and the trustworthy-email guidance of NIST SP 800-177. For organisations that must demonstrate due diligence to insurers or regulators, the ability to show published SPF, DKIM and DMARC records, together with a documented key-management process, is tangible evidence of a mature email-security posture.

7. Conclusion

This project put the practical, server-side pieces of SPF, DKIM and DMARC in place for the laboratory domain lois.com. OpenDKIM was installed, an RSA key pair was generated under the selector "mail", the signing table was configured, and Postfix was confirmed to hand messages to the milter on port 8891. The public key record was extracted and is ready for DNS publication. Complementary SPF and DMARC records were prepared so that the full authentication stack can be switched on once the DNS changes are live.

Two practical lessons stand out. First, even a small path error can stop key generation; the error message was clear and the fix was immediate, but the episode is a reminder to verify directory structure before running tools that expect a particular layout. Second, a local configuration can be perfect and still report "record not found" until the public key appears in DNS. That dependence is not a defect; it is simply the distributed nature of email authentication. Recognising it early prevents wasted debugging time on the server when the real work still lies in the DNS zone.

From a Blue Team standpoint the laboratory supplies a reproducible template that can be applied to any domain for which the organisation controls both the mail server and the DNS zone. The same workflow, once automated, can become part of a standard hardening checklist for new mail domains and can be audited against the CIS mail-server benchmarks and the guidance in NIST SP 800-177. The screenshots, configuration excerpts and command chronology collected during the sessions form a complete evidence package that shows both technical competence and the habit of methodical documentation.

The work is not finished when the last configuration file is saved. The real measure of success will be the Authentication-Results headers that appear on messages leaving the laboratory once the DNS records are live, and the clean aggregate reports that arrive once DMARC reporting is switched on. Those outcomes lie outside the scope of the present sessions, but the server-side foundation required to reach them is now in place and fully documented.

References

[1] S. Kitterman, "Sender Policy Framework (SPF) for Authorizing Use of Domains in Email, Version 1," RFC 7208, IETF, Apr. 2014.

[2] D. Crocker, T. Hansen, and M. Kucherawy, "DomainKeys Identified Mail (DKIM) Signatures," RFC 6376, IETF, Sep. 2011.

[3] M. Kucherawy and E. Zwicky, "Domain-based Message Authentication, Reporting, and Conformance (DMARC)," RFC 7489, IETF, Mar. 2015.

[4] National Institute of Standards and Technology, "Trustworthy Email," NIST Special Publication 800-177 Rev. 1, 2019.

[5] Center for Internet Security, "CIS Benchmarks for Mail Servers," latest available release.

[6] MITRE Corporation, "MITRE ATT&CK: T1566 Phishing," https://attack.mitre.org/techniques/T1566/ (accessed Aug. 2026).

[7] OpenDKIM Project, "OpenDKIM Documentation and Man Pages," http://www.opendkim.org/.

[8] Postfix, "Postfix Milter Support," http://www.postfix.org/MILTER_README.html.

[9] Ubuntu Server Documentation, "Postfix and related mail packages," official community guides.

[10] Messaging, Malware and Mobile Anti-Abuse Working Group (M3AAWG), "DMARC Implementation Best Practices," latest revision.

Appendix A – Full Configuration Excerpts

The excerpts below represent the complete set of configuration artefacts prepared during the laboratory. Only the signing-table line and the Postfix milter settings appear in the live screenshots; the remaining files are the natural companions required for a working OpenDKIM installation.

A.1 /etc/opendkim/signing.table

```
*@lois.com    mail._domainkey.lois.com
```

A.2 /etc/opendkim/key.table (prepared)

```
mail._domainkey.lois.com    lois.com:mail:/etc/opendkim/keys/lois.com/mail.private
```

A.3 /etc/opendkim/TrustedHosts (prepared)

```
127.0.0.1
localhost
MailServerLab
lois.com
*.lois.com
```

A.4 Essential /etc/opendkim.conf directives

```
Syslog                  yes
UMask                   002
Mode                    sv
Canonicalization        relaxed/simple
ExternalIgnoreList      refile:/etc/opendkim/TrustedHosts
InternalHosts           refile:/etc/opendkim/TrustedHosts
KeyTable                refile:/etc/opendkim/key.table
SigningTable            refile:/etc/opendkim/signing.table
SignatureAlgorithm      rsa-sha256
Socket                  inet:8891@localhost
PidFile                 /var/run/opendkim/opendkim.pid
UserID                  opendkim:opendkim
```

A.5 Postfix milter settings (from main.cf)

```
milter_default_action = accept
milter_protocol = 6
smtpd_milters = inet:localhost:8891
non_smtpd_milters = inet:localhost:8891
```

Appendix B – Command Chronology

The list below records the principal commands executed during the laboratory sessions, in approximate chronological order. It is taken directly from the terminal screenshots and can serve as a reproducible checklist for future exercises.

- sudo apt upgrade -y
- sudo apt install git -y (followed by git --version and git config --global ...)
- sudo apt install opendkim opendkim-tools -y
- sudo mkdir -p /etc/opendkim/keys/lois.com
- sudo opendkim-genkey -D /etc/opendkim/keys/lois.com -d lois.com -s mail (after correcting the path)
- ls -l /etc/opendkim/keys/lois.com
- sudo cat /etc/opendkim/keys/lois.com/mail.txt
- sudo nano /etc/opendkim/signing.table
- sudo grep -A5 milter /etc/postfix/main.cf

- opendkim-testkey -d lois.com -s mail -vvv

End of Report